

ASP Encodings for Multi-Network Batching: Naive Baseline and Optimised Encoding

March 2026

1 Problem Overview

The multi-network batching problem requires making three interdependent decisions simultaneously over a logistics network:

- **Flow:** how many units of each product travel along each directed arc,
- **Packing:** how many units of each product are loaded into each bin on each route,
- **Frequency:** how many times each vehicle on each route departs (per bin slot).

The objective is to minimise total transport cost, defined as the sum of frequency $\times$ distance $\times$ cost across all active routes, subject to flow conservation, vehicle capacity, and flow-satisfaction constraints.

2 Naive Baseline Encoding

2.1 Overview

The baseline encoding is a direct ASP translation of the problem. It defines three choice rules that link directly to the flow, packing configuration, and the frequency of transporting items. The constraints then eliminate infeasible combinations.

2.2 Demand and Supply Classification

```
1 netdemandSupply(P,L,0) :- not demandOffer(P,L,_), part(P),  
   location(L).  
2 netdemandSupply(P,L,D) :- demandOffer(P,L,D).  
3  
4 is_consumer(P, L) :- demandOffer(P, L, D), D < 0.  
5 is_producer(P, L) :- demandOffer(P, L, D), D > 0.
```

Listing 1: Demand/supply base facts

Every (product, location) pair is classified as either a consumer, a producer, or neutral. Neutral locations get net demand/supply of zero.

2.3 Decision Variables

The three choice rules define the search space:

```
1 { flow(From,To,Part,N) : numFlow(N) }1 :-  
2   route(From,To,_,_,_),  
3   part(Part),  
4   not is_consumer(Part, From),  
5   not is_producer(Part, To).
```

Listing 2: Flow choice rule

```
1 {pack(N,P,ID,From,To,TR) : numParts(N)} = 1 :-  
2   route(From,To,TR,_,_), bins(ID), part(P).
```

Listing 3: Pack choice rule

```
1 1{freq(ID,From,To,TR,F) : numFreq(F)}1 :-  
2   route(From,To,TR,_,_), bins(ID).
```

Listing 4: Frequency choice rule

The **flow** choice ranges over all values in **numFlow** for every non-trivial (route, product) pair. The **pack** choice ranges over all values in **numParts** for every (route, bin, product) triple. The **freq** choice ranges over all values in **numFreq** for every (route, bin) pair. No domain restriction is applied beyond the basic consumer/producer exclusion.

2.4 Constraints

```
1 :- #sum{ N,To : flow(Loc,To,Part,N);  
2     -N,From : flow(From,Loc,Part,N) } != DS,  
3     netdemandSupply(Part,Loc,DS).
```

Listing 5: Flow conservation constraint

```
1 :- #sum{N*Size : pack(N,P,ID,From,To,TR), partSize(P,Size)} > C,  
2     transportCapacity(TR,C), route(From,To,TR,_,_), bins(ID).
```

Listing 6: Bin capacity constraint

```
1 :- #sum{N*F,ID : pack(N,P,ID,From,To,TR), freq(ID,From,To,TR,F)}  
2     < Value,  
     flow(From,To,P,Value), Value > 0.
```

Listing 7: Flow satisfaction constraint

Flow conservation requires that for every (product, location) the net outflow matches the declared demand/supply value. The capacity constraint prevents a bin from being over-packed. The flow satisfaction constraint ensures that the total units dispatched across all bins on a route (pack count $\times$ frequency, summed over bins) is at least as large as the flow value assigned to that arc.

2.5 Objective

```
1 #minimize{F*Distance*Cost@1,ID,From,To,TR :  
2     freq(ID,From,To,TR,F),  
3     route(From,To,TR,Distance,Cost)}.
```

Listing 8: Minimisation objective

Total cost is minimised over the product of frequency, distance, and per-unit cost for every active (route, bin) pair. For the time being we only considered this metric in optimising the cost of the network.

3 Optimised Encoding

3.1 Overview

The optimised encoding applies four structural improvements to the baseline. These target the search space directly: tightening domains that the choice rules can generate rules from, pruning unreachable routes, and eliminating bin permutation symmetry.

3.2 Optimisation 1: Domain Tightening

3.2.1 Flow Domain

The baseline uses a global `numFlow` domain for all flow variables. The optimised encoding replaces this with a per-product tight bound: flow on any single arc cannot exceed the total supply of that product across the entire network.

```
1 totalSupply(P, S) :-  
2     S = #sum{ D,L : demandOffer(P,L,D), D > 0 },  
3     part(P).  
4  
5 validFlow(P, 0) :- part(P).  
6 validFlow(P, N) :- numFlow(N), totalSupply(P, S), N <= S.
```

Listing 9: Total supply computation and valid flow domain

A flow value of 100 units on an arc is infeasible if only 5 units of that product exist in the entire network. The baseline would still generate atoms for those values and branch over them.

3.2.2 Pack Domain

The maximum number of units of product P that can fit in a single bin on transport TR is bounded by $\lfloor C/\text{size}(P) \rfloor$, where C is the vehicle capacity.

```
1 maxPack(P, TR, M) :-  
2     partSize(P, Size), Size > 0,  
3     transportCapacity(TR, C),  
4     M = C / Size.  
5  
6 validPack(P, TR, 0) :- part(P), transportCapacity(TR, _).
```

```

7 validPack(P, TR, N) :-
8     numParts(N), maxPack(P, TR, M), N <= M.

```

Listing 10: Pack domain bounded by physical capacity

For a vehicle with capacity 10 and a part of size 4, the maximum pack count is 2. The baseline creates atoms `pack(3,...)`, `pack(4,...)` and so on, and generates constraints for them; the optimised encoding simply never creates those atoms. This is most effective when parts are large relative to vehicle capacity.

3.3 Optimisation 2: Route Pruning via Reachability Analysis

The baseline generates flow variables for every (route, product) pair that is not immediately excluded by the consumer/producer check. Many of these pairs are structurally impossible: supply of product P can never reach location `From` given the network topology.

The optimised encoding computes forward reachability (can supply reach a location?) and backward reachability (is there downstream demand reachable from a location?) before the solve phase.

```

1 % Forward: locations reachable from producers
2 can_supply(P, L) :- is_producer(P, L).
3 can_supply(P, To) :- can_supply(P, From), route(From, To, _, _,
4     _).
5 % Backward: locations from which consumers are reachable
6 can_receive(P, L) :- is_consumer(P, L).
7 can_receive(P, From) :- can_receive(P, To), route(From, To, _, _
8     , _).
9 % A route is active only if both conditions hold
10 active_route(From, To, P) :-
11     route(From, To, _, _, _), part(P),
12     can_supply(P, From),
13     can_receive(P, To),
14     not is_consumer(P, From),
15     not is_producer(P, To).

```

Listing 11: Reachability computation and active route derivation

Flow choice rules are only generated for `active_route` pairs. Inactive routes receive `flow = 0` by deterministic derivation, not by solver search, removing entire branches from the search tree before solving begins.

```

1 1{ flow(From, To, Part, N) : validFlow(Part, N) }1 :-
2     active_route(From, To, Part).
3
4 flow(From, To, Part, 0) :-
5     route(From, To, _, _, _), part(Part),
6     not is_consumer(Part, From),
7     not is_producer(Part, To),
8     not active_route(From, To, Part).

```

3.4 Optimisation 3: Supply-Based Packing Constraint

Even after route pruning, the `pack` choice rule still generates atoms for all (route, bin, product) triples. A hard constraint eliminates packing on any route from which supply of that product cannot arrive:

```

1 :- pack(N, P, ID, From, To, TR), N > 0,
2    not can_supply(P, From).
```

Listing 13: Supply-based packing constraint

The key property of this constraint is *when* it fires. Because `can_supply` is derived deterministically during grounding, this integrity constraint is checked during unit propagation - the cheap, deterministic inference phase that runs before each choice. The moment the solver tentatively assigns `pack > 0` to an unreachable location, the constraint fires immediately. The baseline's equivalent detection requires flow conservation to eventually derive a contradiction, which only happens after several additional downstream assignments have been made. This accelerates conflict detection and reduces the number of choices made per conflict.

3.5 Optimisation 4: Symmetry Breaking on Bins

3.5.1 The Symmetry Problem

Bins are interchangeable labels. If a solution assigns frequency f_1 to bin 1 and f_2 to bin 2 on route r , then swapping the bin labels produces a solution with identical cost and feasibility. With B bins there are $B!$ such permutations per route. The baseline explores all of them, which causes search cost to grow approximately as $O(B!)$ in the bin dimension. At $B = 3$ this is a $6\times$ redundancy; at $B = 4$ it would be $24\times$.

3.5.2 Primary Ordering by Frequency

Bins are ordered by non-increasing frequency across all routes. This eliminates $B! - 1$ of the $B!$ permutations.

```

1 :- freq(ID, From, To, TR, F1),
2    freq(ID+1, From, To, TR, F2),
3    F1 < F2,
4    bins(ID), bins(ID+1),
5    route(From, To, TR, _, _).
```

Listing 14: SB4a: non-increasing frequency order across consecutive bins

3.5.3 Secondary Tiebreak by Pack Count

When two consecutive bins carry the same frequency, residual symmetry remains. SB4b canonicalises this by requiring non-increasing pack counts for equal-frequency bins:

```

1 :- freq(ID, From, To, TR, F),
2   freq(ID+1, From, To, TR, F),
3   pack(N1, P, ID, From, To, TR),
4   pack(N2, P, ID+1, From, To, TR),
5   bins(ID), bins(ID+1),
6   route(From, To, TR, _, _), part(P),
7   N1 < N2.

```

Listing 15: SB4b: tiebreak on pack count for equal-frequency bins

3.5.4 Forced Zero-Packing on Empty Bins

If a bin is assigned zero frequency it carries no goods. Rather than leaving this to constraint propagation, the zero packing is derived directly:

```

1 pack(0, P, ID, From, To, TR) :-
2   freq(ID, From, To, TR, 0),
3   route(From, To, TR, _, _), bins(ID), part(P).

```

Listing 16: SB4c: deterministic zero-pack for zero-frequency bins

Because this is a derived fact, the solver never branches on `pack` variables for empty bins. In the baseline, a bin with `freq = 0` still has pack atoms that the solver must assign and verify.

4 Search Space Analysis

This section derives the theoretical search space for each encoding and works through a concrete example using a real instance, then compares the two encodings at each bin count.

4.1 General Formula

Let:

- R = total number of directed route arcs in the instance
- R_f = number of route arcs with non-trivial flow (after consumer/producer exclusion)
- R_a = number of *active* routes for a product (after reachability pruning, $R_a \leq R_f$)
- P = number of products
- B = number of bins
- $|numFlow|$ = size of the flow integer domain ($= S_{\max} + 1$ where $S_{\max}$ is total supply)
- $|numParts|$ = size of the pack integer domain ($= \lfloor C/s \rfloor + 1$ per transport and part)
- $|numFreq|$ = size of the frequency domain

4.1.1 Baseline

The baseline creates one choice variable per slot in the following three families:

Flow atoms: $R_f \times P$ variables, domain size $|numFlow|$
 Pack atoms: $R \times B \times P$ variables, domain size $|numParts|$
 Freq atoms: $R \times B$ variables, domain size $|numFreq|$

The theoretical upper bound on the search space (before any constraint propagation) is:

$$\mathcal{S}_{\text{baseline}} = |numFlow|^{R_f \cdot P} \times |numParts|^{R \cdot B \cdot P} \times |numFreq|^{R \cdot B} \quad (1)$$

4.1.2 Optimised

Each optimisation reduces a factor in Equation 1:

- **Opt 1** replaces $|numFlow|$ with $|validFlow(P)| = \min(S_P, S_{\max}) + 1$ and $|numParts|$ with $|validPack(P, TR)| = \lfloor C_{TR}/s_P \rfloor + 1$.
- **Opt 2** replaces R_f with $R_a \leq R_f$ for the flow dimension (inactive routes are fixed to zero by derivation).
- **Opt 4 (SB4a)** constrains frequency assignments to be non-increasing across bins. For B bins with domain of size D , the number of valid non-increasing sequences is $\binom{D+B-1}{B}$ rather than D^B . The reduction factor per route is:

$$\frac{D^B}{\binom{D+B-1}{B}}$$

which approaches $B!$ as $D \rightarrow \infty$.

The optimised search space is therefore:

$$\mathcal{S}_{\text{opt}} = |validFlow|^{R_a \cdot P} \times |validPack|^{R \cdot B \cdot P} \times \binom{|numFreq| + B - 1}{B}^R \quad (2)$$

5 Experiments

5.1 Instance Generation

Instances were generated using a Python random instance generator.

5.2 Instance Series

Series	Varying	Fixed	Purpose
A	$L \in \{4, 6, 8, 10\}$	$P = 1, \text{TR} = 2, B = 1$	Location scaling
B	$P \in \{1, 2, 3\}$	$L = 6, \text{TR} = 2, B = 1$	Product scaling
C	$\text{TR} \in \{1, 2, 3\}$	$L = 6, P = 2, B = 1$	Transport scaling
D	$B \in \{1, 2, 3\}$	$L = 6, P = 2, \text{TR} = 2$	Bin scaling
E	Capacity tightness	$L = 6, P = 2, \text{TR} = 2, B = 1$	Capacity regime
F	$L \in \{4, 6, 8, 10, 12, 15\}$	$P = 2, \text{TR} = 2, B = 2$	Extended location scaling
G	$P \in \{1, 2, 3, 4, 5\}$	$L = 8, \text{TR} = 2, B = 2$	Extended product scaling

Table 1: Instance series. L = locations, P = products, TR = transport types, B = bins.

Capacity regimes for Series E:

- **Tight:** capacity = 10, part size = 4
- **Medium:** capacity = 15, part size = 3
- **Loose:** capacity = 20, part size = 1

5.3 Solver Configurations

All runs used clingo with a time limit of 300 seconds. Four configurations were tested:

Configuration	Character
<code>jumpy</code>	Aggressive restarts; default heuristic
<code>tweety</code>	Industrial heuristic; Luby-schedule restarts
<code>handy</code>	Conservative, low-restart strategy; large-problem defaults
<code>parallel</code>	4-thread portfolio (<code>-parallel-mode=4</code>)

Table 2: Clingo solver configurations used in the configuration sweep.

5.4 Metrics

Metric	Meaning
<code>atoms / rules</code>	Grounded program size
<code>choices</code>	Branching decisions; primary measure of search effort
<code>conflicts</code>	Contradictions encountered; triggers clause learning
<code>restarts</code>	Solver heuristic restarts
<code>ground_time_s</code>	Wall-clock time in grounding phase
<code>solve_time_s</code>	Wall-clock time in solver search
<code>result</code>	OPTIMUM, SAT_TIMEOUT, TIMEOUT, UNSAT, ERROR

Table 3: Metrics recorded per run.

6 Results

6.1 Grounding vs. Solving: Where Is the Time Spent?

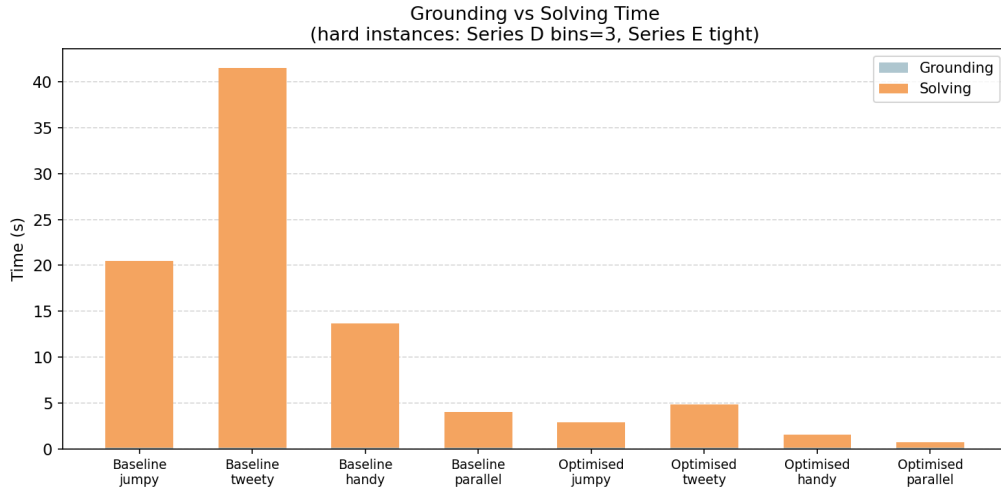


Figure 1: Grounding time versus solve time across regimes. Grounding is negligible at all difficulty levels.

Regime	Avg ground (s)	Avg solve (s)	Ground %
Series D, $B = 1$	0.05	0.25	17%
Series D, $B = 2$	0.10	3.5	3%
Series D, $B = 3$	0.19	28	<1%
Series E, tight	0.03	3.4	1%

Table 4: Grounding time as a fraction of total time. Solving dominates at every difficulty level.

Grounding time grows roughly linearly with program size while solve time grows significantly more. We can see that most of the runtime spent for finding a solution is in the grounding.

6.2 Choices Are Proportional to Solve Time

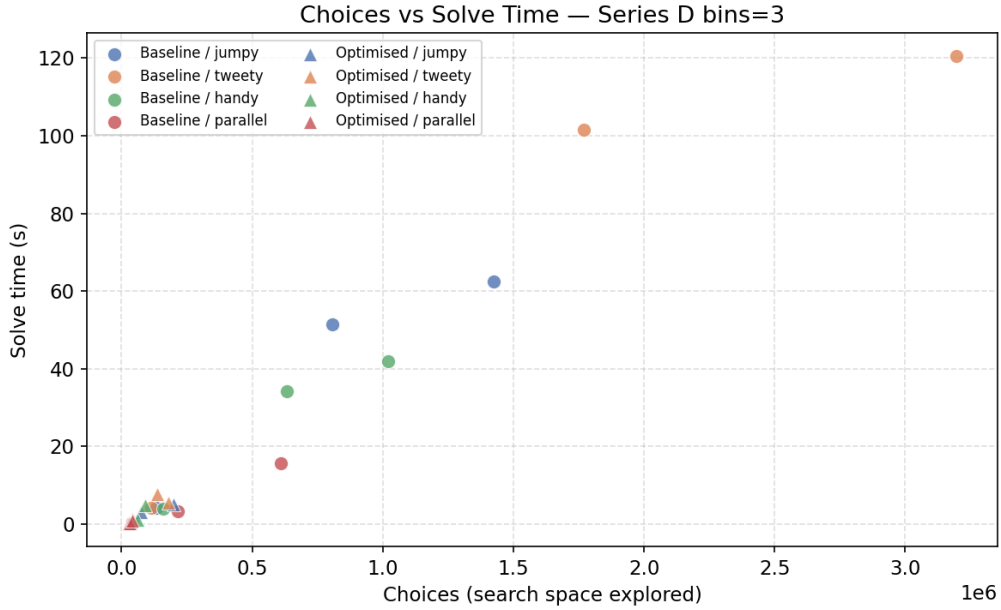


Figure 2: Choices vs. solve time across all Series D bins=3 runs. Each point is one run.

The scatter plot confirms near-linear proportionality between choices and solve time across all encodings and configurations. The solver spends approximately constant time processing each branching decision, so choices is a reliable dimensionless proxy for search effort.

Notably, all optimised encoding runs cluster near the origin regardless of configuration. All baseline runs are scattered across a much wider range, with the worst cases reaching 3.2 million choices and 120 seconds. The horizontal gap between the two groups is a direct visual representation of the speedup. This speedup is most likely due to the bin packing symmetry breaking.

6.3 Series D: Bin Scaling

Series D is the hardest instance series and the primary motivation for the optimised encoding. The symmetry breaking targets the bin packing directly.

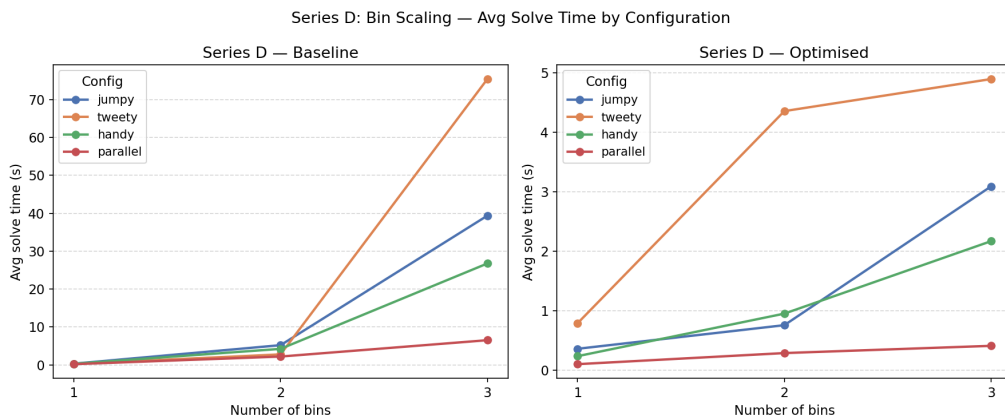


Figure 3: Mean solve time vs. bin count by encoding and configuration (Series D).

Encoding	Config	$B = 1$ avg (s)	$B = 3$ avg (s)	Growth factor
encoding_route	tweety	0.25	75.4	302×
encoding_route	jumpy	0.31	39.5	127×
encoding_route	handy	0.15	26.9	178×
encoding_route	parallel	0.19	6.7	35×
optimised_encoding	tweety	0.46	5.1	11×
optimised_encoding	jumpy	0.36	3.2	9×
optimised_encoding	handy	0.16	2.3	14×
optimised_encoding	parallel	0.08	0.6	7×

Table 5: Solve time growth from $B = 1$ to $B = 3$ for each encoding and configuration.

The baseline grows 35-302× going from $B = 1$ to $B = 3$. The optimised encoding grows only 7-14×. This difference in growth rate reflects the bin permutation symmetry: the baseline scales approximately as $O(B!)$ while the optimised encoding scales much more gently due to symmetry breaking.

6.4 Conflict Rate Analysis

Instance	Encoding	Config	Choices	Conflicts	Conflict rate
s2	encoding_route	tweety	3,195,652	758,932	23.7%
s2	encoding_route	handy	1,019,054	265,457	26.1%
s2	optimised_encoding	tweety	180,439	15,051	8.3%
s2	optimised_encoding	handy	33,343	2,824	8.5%
s3	encoding_route	tweety	1,769,212	342,246	19.4%
s3	optimised_encoding	tweety	136,029	19,242	14.1%
tight_s3	encoding_route	tweety	557,548	264,664	47.5%
tight_s3	optimised_encoding	tweety	422,541	194,102	45.9%
tight_s3	encoding_route	handy	34,950	10,100	28.9%
tight_s3	optimised_encoding	handy	86,136	26,574	30.9%

Table 6: Choices, conflicts, and conflict rates. Top rows: Series D, $B = 3$. Bottom rows: Series E, tight capacity.

The baseline has a 2-3× higher conflict rate than the optimised encoding at $B = 3$. This shows that the baseline makes more choices before each conflict. The optimised encoding’s constraints fire during unit propagation on the very first bad assignment, so conflicts are found earlier and each backtrack discards a shorter chain of work. This guides the solver better.

6.5 Grounded Program Size

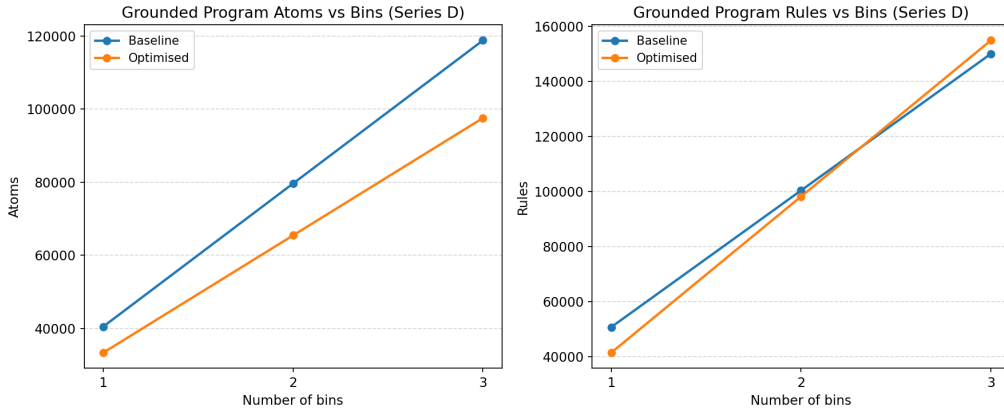


Figure 4: Grounded program size (atoms and rules) by bin count and encoding.

Encoding	$B = 1$ atoms	$B = 3$ atoms	$B = 1$ rules	$B = 3$ rules
encoding_route	$\approx 40,500$	$\approx 118,800$	$\approx 60,000$	$\approx 176,000$
optimised_encoding	$\approx 33,400$	$\approx 97,600$	$\approx 50,000$	$\approx 156,000$

Table 7: Grounded program sizes at $B = 1$ and $B = 3$.

The optimised encoding is approximately 18% smaller at $B = 3$. Program size reduction is a side effect of route pruning.